Air Quality Clustering

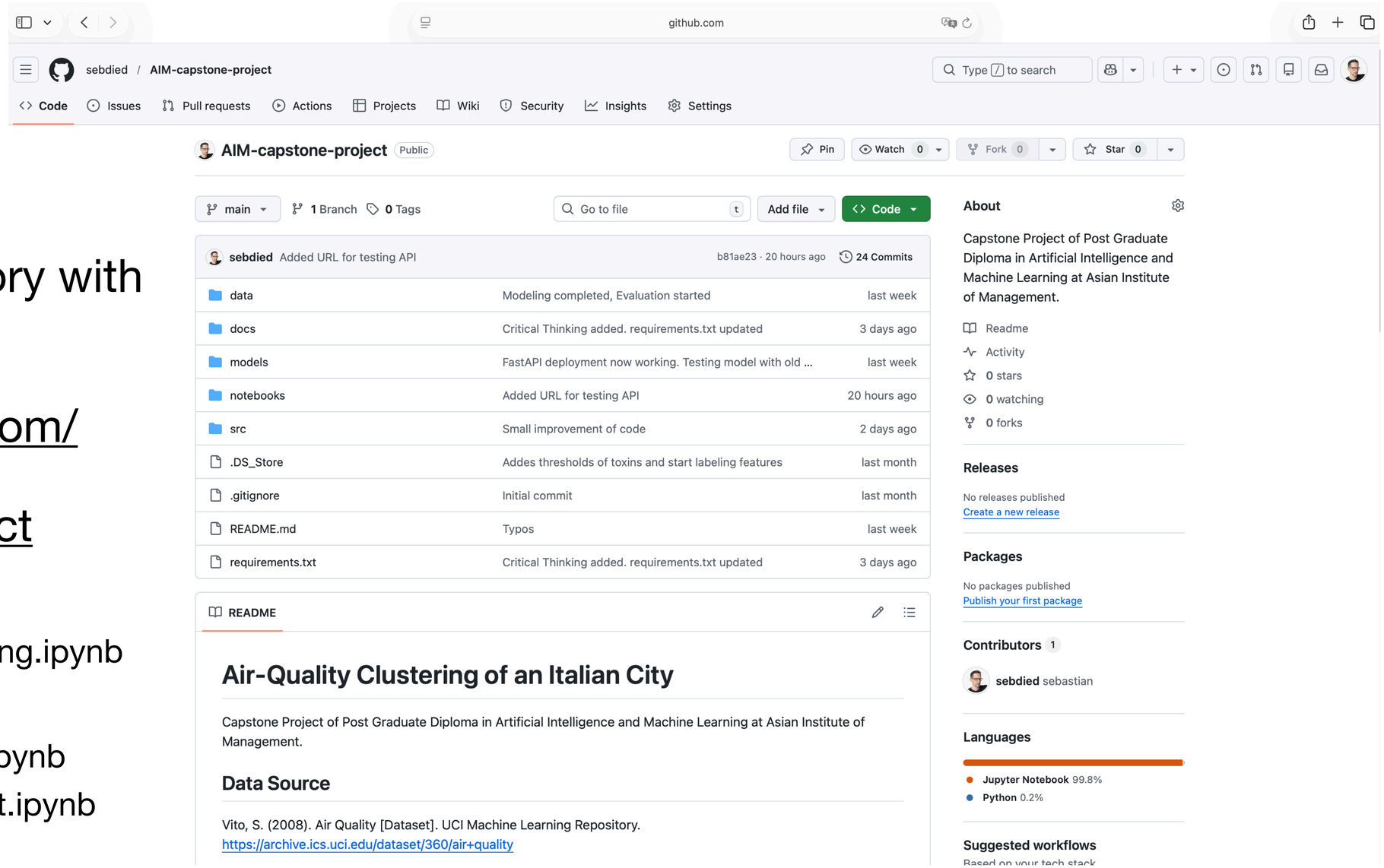
Grouping Air Quality Patterns Across 12 Months in an Italian City

Dr. Sebastian Diedrich - March 2026

- GitHub repository with **all** Files:

https://github.com/sebdied/AIM-capstone-project

01_EDA.ipynb
02_Feature_Engineering.ipynb
03_Modeling.ipynb
04_Evaluation.ipynb
05_Critical_Thinking.ipynb
07_Local_Deployment.ipynb



Problem

Air Quality and Health Issues

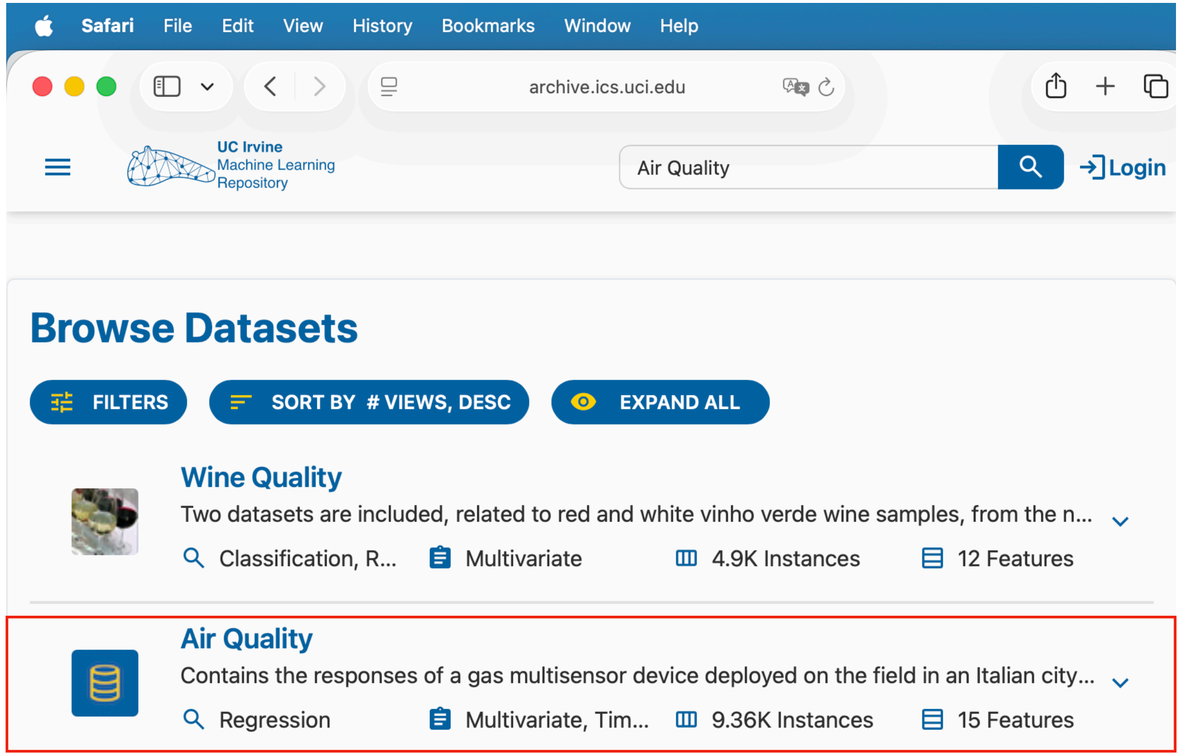
- Task: Clustering
- Technical Success Metric:
  Silhouette Score > 0.5
- Business Success Metric:
  Cluster of 3 groups:
  - Green: Low pollution
  - Yellow: Moderate pollution
  - Red: Severe pollution
- Goal: Reducing healthcare costs
- ./notebooks/**01_EDA.ipynb**

AI generated

Data Collection

UCI

- Source: https://archive.ics.uci.edu/dataset/360/air+quality Vito, S. (2008)
- Instances of Real Data: 9,358 of Chemical Multisensor Devices
- Data recorded from March 2004 to February 2005 (one year)
- 15 Features
- ./notebooks/**01_EDA.ipynb**



Data Collection

Data Dictionary

- Types: object and float64, Missing values: yes, error code always ‘-200’
- ./docs/**DataDictionary.txt**

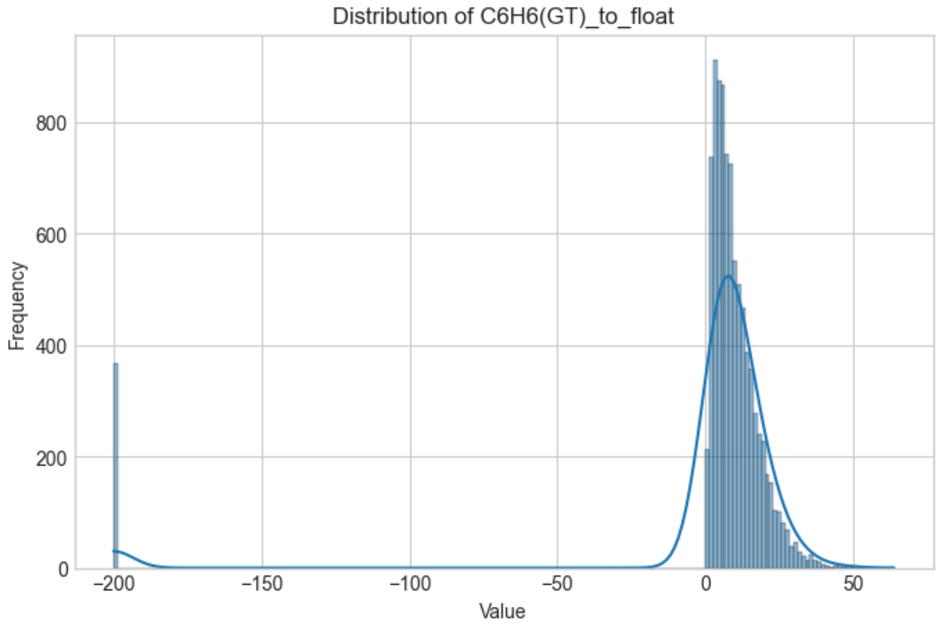
Column Index	Column Name	Type	Info	Allowed Values
0	Date	object	Date with format: DD/MM/YYYY	DD/MM/YYYY
1	Time	object	Time with format: HH.MM.SS	HH.MM.SS
2	CO(GT)	object	CO concencration in mg/m^3	>= 0 or -200 (error)
3	PT08.S1(CO)	float64	Hourly averaged sensor response (nominally CO targeted)	>= 0 or -200 (error)
4	NMHC(GT)	float64	Non Metanic HydroCarbons concentration in microg/m^3'	>= 0 or -200 (error)
5	C6H6(GT)	object	Benzene concentration in microg/m^3	>= 0 or -200 (error)
6	PT08.S2(NMHC)	float64	Hourly averaged sensor response (nominally NMHC targeted)	>= 0 or -200 (error)
7	NOx(GT)	float64	NOx concentration in ppb	>= 0 or -200 (error)
8	PT08.S3(NOx)	float64	Hourly averaged sensor response (nominally NO2 targeted)	>= 0 or -200 (error)
9	NO2(GT)	float64	NO2 concentration in microg/m^3	>= 0 or -200 (error)
10	PT08.S4(NO2)	float64	Hourly averaged sensor response (nominally NO2 targeted)	>= 0 or -200 (error)
11	PT08.S5(O3)	float64	Ozon (O3)	>= 0 or -200 (error)
12	T	object	Temperature (°C)	-50 >= 0 or -200 (error)
13	RH	object	Relative Humidity (%)	0-100 or -200 (error)
14	AH	object	Absolute Humidity	>= 0 or -200 (error)

Preprocessing

Cleaning and handling nulls (1)

- Every column was analysed by using the function: *analyze_col(…)*

```python
# Analyse data distribution and neg. values of a column
def analyze_col(df: pd.DataFrame, col_name: str, convert_dtype_to_float: bool):
    print(f"# Column name: {col_name}")
    if col_name in df.columns:
        # Types in column
        print(f'Types found ({col_name}): {df[col_name].dtypes}')
        # Try to convert dtype to float to check for neg. values
        if convert_dtype_to_float is True:
            try:
                # Convert the column to numeric, handling errors
                # Replace ',' with '.' if necessary and convert to float
                col_to_float = f'{col_name}_to_float'
                df[col_to_float] = df[col_name].str.replace(',', '.', regex=False).astype(float)
                print(f'Types found ({col_to_float}): {df[col_to_float].dtypes}')
                # Count of neg. values
                print(f'Neg. values ({col_to_float}): {(df[col_to_float] < 0).sum()}')
                # Plot distribution
                plot_distributions(df, col_to_float, (15, 5))
            except Exception as ex:
                print(f"Could not convert column-dtype to float: {ex}")
        else:
            # Count of neg. values
            print(f'Neg. values ({col_name}): {(df[col_name] < 0).sum()}')
            # Plot distribution
            plot_distributions(df, [col_name], (15, 5))
```



Values were casted to float-dtype for plotting

- Outliers or error readings (-200) could easily be found
- ./notebooks/**01_EDA.ipynb**

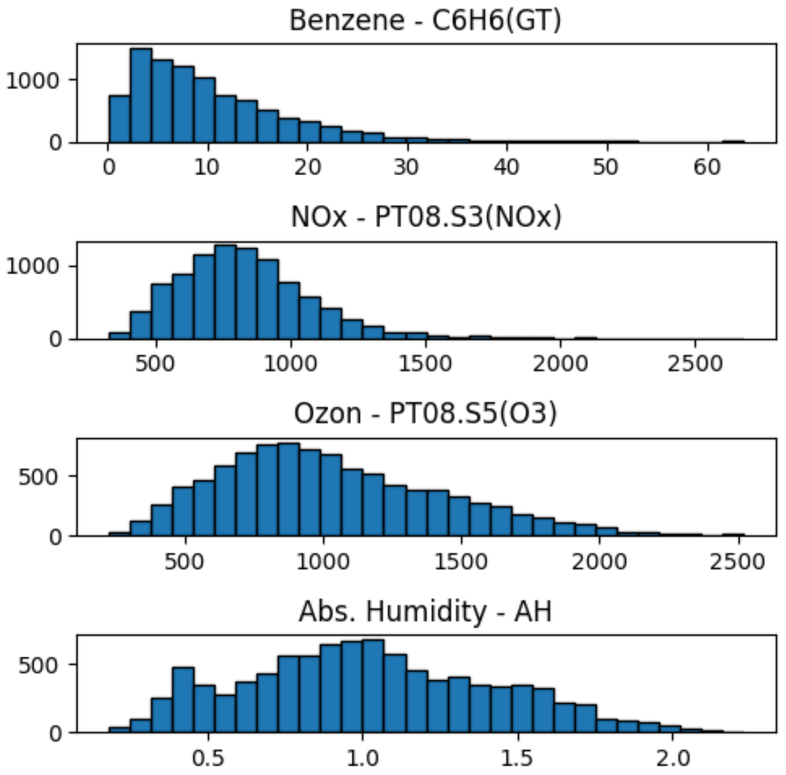
Preprocessing

Cleaning and handling nulls (2)

- All error readings (-200) were set to NaN
- These NaN were replaced by interpolation using the data points before and after the timestamp(s)
- Result: numerical type, no nulls and meaningful interpolated data points

```
# Clean and transform column: 'C6H6(GT)'
# Replace each '-200' with NaN and interpolate the missing value(s)
df['C6H6(GT)_to_float'] = df['C6H6(GT)'].str.replace(',', '.', regex=False).astype(float) # 0,4 -> 0.4
df['C6H6(GT)_to_float'] = df['C6H6(GT)_to_float'].replace(-200, np.nan)
df['C6H6(GT)'] = df['C6H6(GT)_to_float']
df = df.drop(columns=['C6H6(GT)_to_float'])
df['C6H6(GT)'] = df['C6H6(GT)'].interpolate(method='linear', limit_direction='both')
df['C6H6(GT)'] = df['C6H6(GT)'].round(2) # Round column to 2 decimal places
```

- ./notebooks/**02_Feature_Engineering.ipynb**



Distribution after cleaning and interpolation

Feature Engineering

Creation and Selection

- The columns *Date* and *Time* were handled a little bit differently
- Both had the type: object
- For better calculation I merged both to the new column: DateTime - with the format ‘YYY-MM-DD HH:MM:SS’
- From this new column two other new features were created: *Month* and *HourOfDay*
- If a threshold (WHO) was found for an pollutant, a new column was created, e.g. *C6H6(GT)ToHigh (type: boolean)*
- ./notebooks/**02_Feature_Engineering.ipynb**

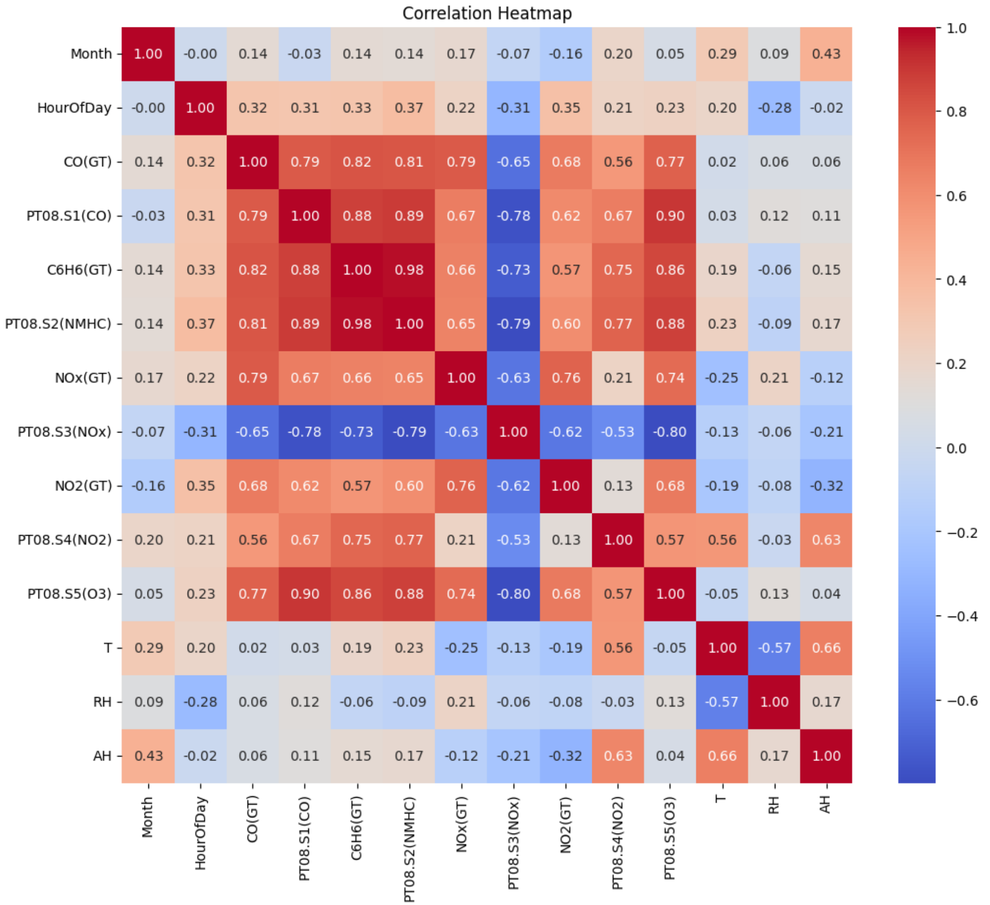
```python
def get_threshold(col_name: str) -> int | None:
    # Thresholds for each toxin by 'WHO Air Quality Guidlines (AQG)' or 'European Environme
    # Source of comment behind each toxin: https://archive.ics.uci.edu/dataset/360/air+qual
    thresholds: dict = {
        'CO(GT)': 4,            # True hourly averaged concentration CO in mg/m^3
        'NMHC(GT)': None,       # True hourly averaged overall Non Metanic HydroCarbons con
        'C6H6(GT)': 5,          # True hourly averaged Benzene concentration in microg/m^3
        'NOx(GT)': None,        # True hourly averaged NOx concentration in ppb
        'NO2(GT)': 10,          # True hourly averaged NO2 concentration in microg/m^3
        'PT08.S5(O3)': 60      # Hourly averaged sensor response (nominally O3 targeted)
    }
    if col_name in thresholds:
        return thresholds[col_name]
    else:
        return None
```

Helper function to get thresholds

Feature Engineering

Creation and Selection

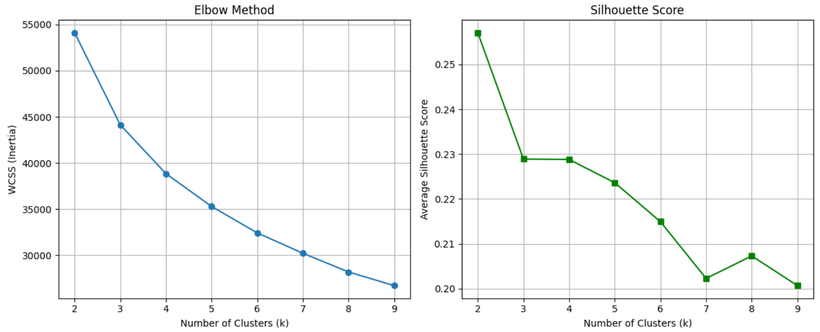
- A Correlation Heatmap was plotted to make a better decision about feature selection
- The feature ‘NMHC(GT)’ was already removed from the dataset because of >90% error readings (interpolation did not make sense)
- The Heatmap interpretation already indicates a stronger correlation among the pollutants themselves than with seasonal factors such as temperature or month
- Relative humidity (RH) was selected over absolute humidity (AH) because it is more commonly used to describe weather conditions.
- ./notebooks/**03_Modeling.ipynb**



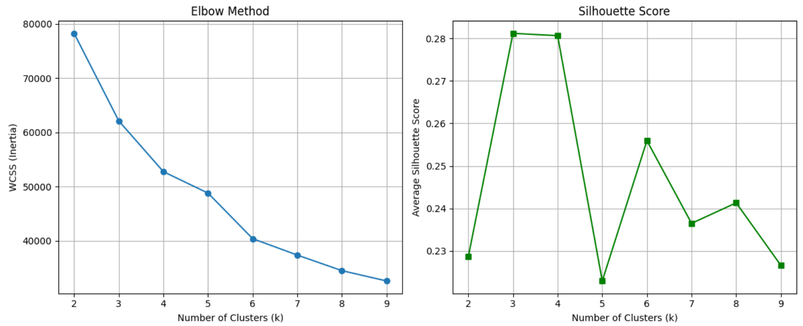
Modeling

Unsupervised: Clustering

- I decided to train the model with two versions of the dataset:
  a) Without the threshold features
  b) With the threshold features
- First attempt was using: K-Means
- Using Elbow Method and Silhouette Score to find the right parameters
- Chose k=3 for training
- ./notebooks/**03_Modeling.ipynb**



a) 'Raw' data from UCI

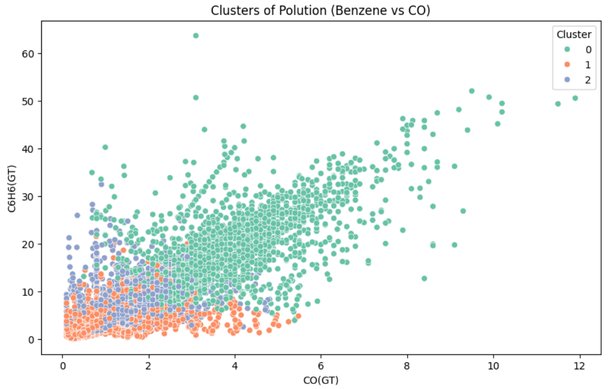


b) Threshold features included

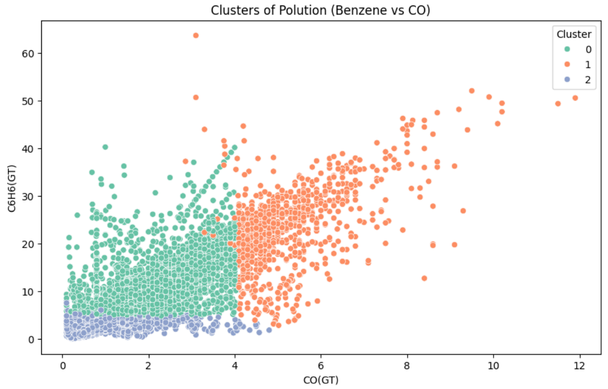
Modeling

Clustering Results

- K-Means could find 3 Clusters in both versions
- The Silhouette Score for k=3 was poor:
  a) 0.23
  b) 0.28
- The plotting on the right shows the strong correlation between Benzene and CO
- Another attempt with DBCAN was made
- ./notebooks/**03_Modeling.ipynb**



a) 'Raw' data from UCI

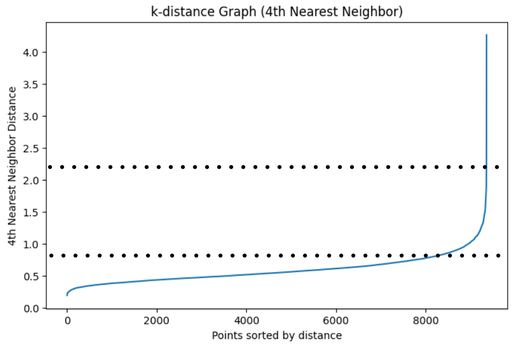


b) Threshold features included

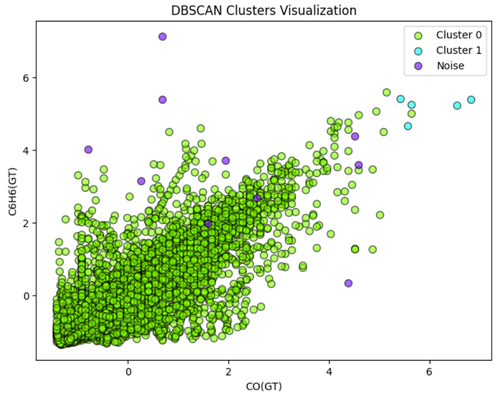
Modeling

Trying another clustering method

- DBSCAN
- Using k-distance graph to choose a good **eps** (4th Nearest Neighbour)
- Experimenting with eps between 1-3 and min-samples-values in a range of 2 to 5
- DBSCAN visualization plot showed 2 clusters and a little bit noise. The green coloured one dominating by over 95%
- Silhouette Score was > 0.5
- ./notebooks/**03_Modeling.ipynb**



The curve is fairly flat until around ~8500



DBSCAN Visualization Plot

Evaluation

Clustering results

- Comparison K-Means vs DBSCAN
  K-Means: 3 clusters (forced)
  DBSCAN: 2 clusters + noise
- The density pattern indicates two main groups. However, the DBSCAN plot shows one dominant large cluster alongside a much smaller secondary cluster.
- The K-Means clustering method was able to identify three clusters, achieving the initial goal of distinguishing low, moderate, and high pollution density levels.
- ./notebooks/**04_Evaluation.ipynb**

```
Cluster Counts:
Cluster
1    3544
2    3318
0    2495
Name: count, dtype: int64
```

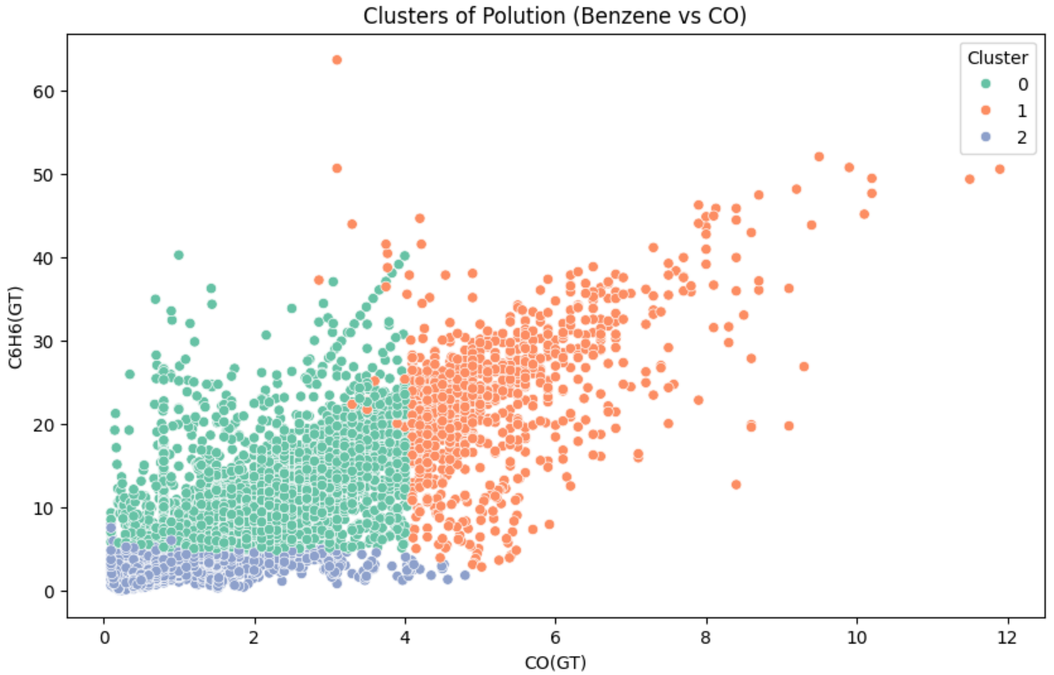
Cluster	Label	CO(GT)	C6H6(GT)	NO2(GT)	PT08.S5(O3)	Hour	Temp (°C)
0	High Pollution / Peak Activity	3.86	19.12	155.98	1541.78	14.10	16.32
1	Low Pollution / Early Morning Clean Air	1.21	4.67	81.41	793.02	5.27	14.10
2	Moderate Pollution / Warm Dry Afternoon	1.82	9.34	104.92	905.46	16.19	24.09

Interpretation of the identified clusters using ChatGPT - K-Means (without predefined pollutant thresholds)

Evaluation

With or Without threshold features?

- K-Means was able to identify more distinct clusters when threshold features were included
- You can easily spot where the toxin levels surpassed the threshold limits.
- The Silhouette Score was still poor with 0.28, indicating that the data set even with these added features is not easy to cluster
- ./notebooks/**04_Evaluation.ipynb**



Evaluation

Overall insights

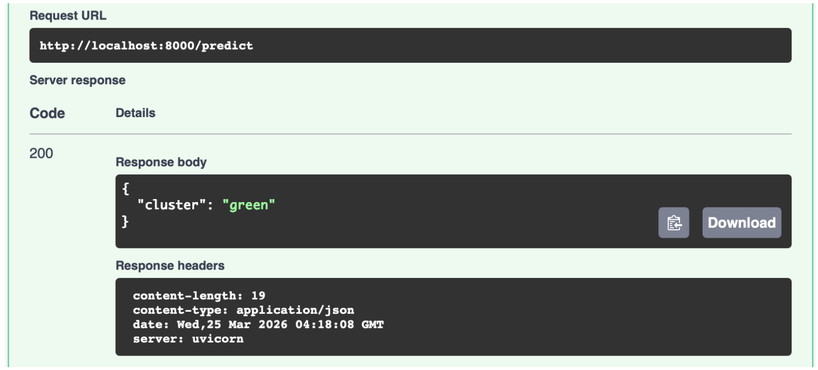
- The technical success criterion-achieving a Silhouette Score ≥ 0.5 - was not met. Observed scores in the range of 0.23 to 0.28 suggest that the dataset does not naturally form well-separated clusters. Incorporating toxin threshold features provided only a marginal improvement.
- The business objective, however, was achieved. Despite the lack of strong natural clustering, the data could be effectively partitioned into three groups using k-means for clustering, enabling a practical "traffic light" classification (low, moderate, extreme) for decision-making.
- DBSCAN proved unsuitable for this dataset, as the observations are densely distributed with no clear gaps. Additionally, variables such as temperature, humidity or month do not exert sufficient influence on pollution levels to create distinct density-based clusters.
- ./notebooks/**04_Evaluation.ipynb**
- Both clustering approaches consistently indicate that morning hours are the most suitable for outdoor sports, while the period between 1 PM and 4 PM should be avoided due to elevated toxin levels.
- Surprisingly, the data does not reveal any strong seasonal patterns or significant weather-driven variation in toxin concentrations.
- The PCA results indicate that the toxin variables themselves have the strongest influence on the clustering. Many pollutants increase simultaneously, highlighting a high degree of multicollinearity in the data.

Deployment

Running test API locally

- Goal: Load saved **model** / **scaler** and run some test api calls
- FastAPI was used to run a Server

```python
class InputData(BaseModel):
    data_row: List[float]

app = FastAPI()
model = joblib.load("../models/kmeans.pkl")
scaler = joblib.load("../models/scaler.pkl")
features: list = pickle.load(open('../data/features.pkl', 'rb'))

@app.post("/predict")
def predict(input: InputData):

    # Convert to DataFrame with correct feature names
    X = pd.DataFrame([input.data_row], columns=features)

    # Ensure numeric (handles booleans too)
    X = X.astype(float)

    # Scale (DO NOT refit)
    X_scaled = scaler.transform(X)

    # Predict
    cluster = model.predict(X_scaled)

    return {"cluster": get_color(int(cluster[0]))}

config = uvicorn.Config(app, host="0.0.0.0", port=8000)
server = uvicorn.Server(config)

await server.serve()
```



- ./notebooks/**07_Deployment.ipynb**

The **/predict** endpoint answered with a **json** response: *cluster -> green*